

Визуализация данных

Лабораторная работа посвящена работе с библиотеками matplotlib и seaborn, позволяющими организовать построение графиков для имеющихся данных.

Рекомендуется выполнять лабораторную работу в Google Colab с использованием браузера. Если на вашем компьютере установлена Anaconda, вы можете скачать задание и выполнять его с использованием Anaconda.

В самом начале работы следует подключить нужные нам библиотеки

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
%matplotlib inline
```

Для выполнения последующих заданий в Google Colab потребуется подключить работу с гугл-диск, запустив следующую ячейку.

(https://colab.research.google.com/drive/1DOwvT_j7OIkJZoNL4LwbXxDwmYUn0TLm?usp=sharing).

Внимание! Если вы выполняете работу в Anaconda, следующую ячейку запускать не нужно!

```
In [3]: # Я работаю в Anaconda
# from google.colab import drive
# drive.mount('/content/drive')
```

Перед выполнением следующих заданий вам нужно загрузить на свой гугл-диск файлы `pogoda.csv`, `tips.csv` и `Video_Games_Sales_as_at_22_Dec_2016.csv`. Файлы прикреплены к заданию.

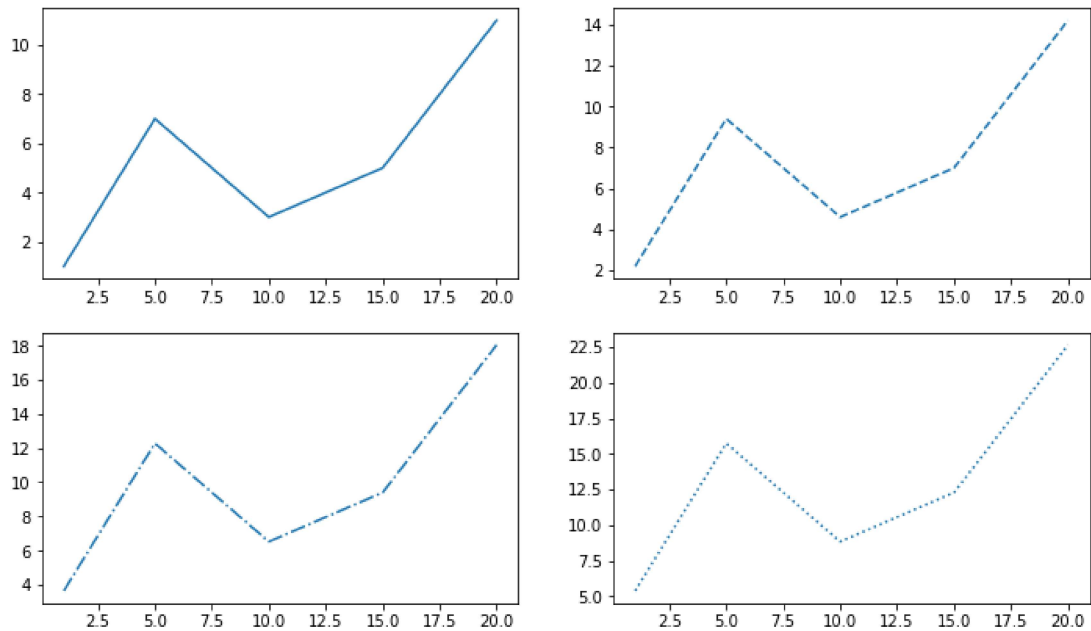
Если вы работаете в Anaconda, файлы нужно скачать на компьютер и поместить в папку, из которой вам будет удобно с ними работать.

Задание 1 Загрузите набор данных `pogoda.csv` и для каждого параметра постройте график. Требования:

- Цвет, стиль, толщина линии у графиков не должны повторяться.
- Каждый график должен быть в своей системе координат, как показано на рисунке 1.
- Добавьте названия, подписи осей и легенду.

Примечание: обратите внимание, что в таблице столбцы разделены символом табуляции. Для открытия необходимо использовать параметр `sep = '\t'`.

Рисунок 1 - Образец оформления графиков.



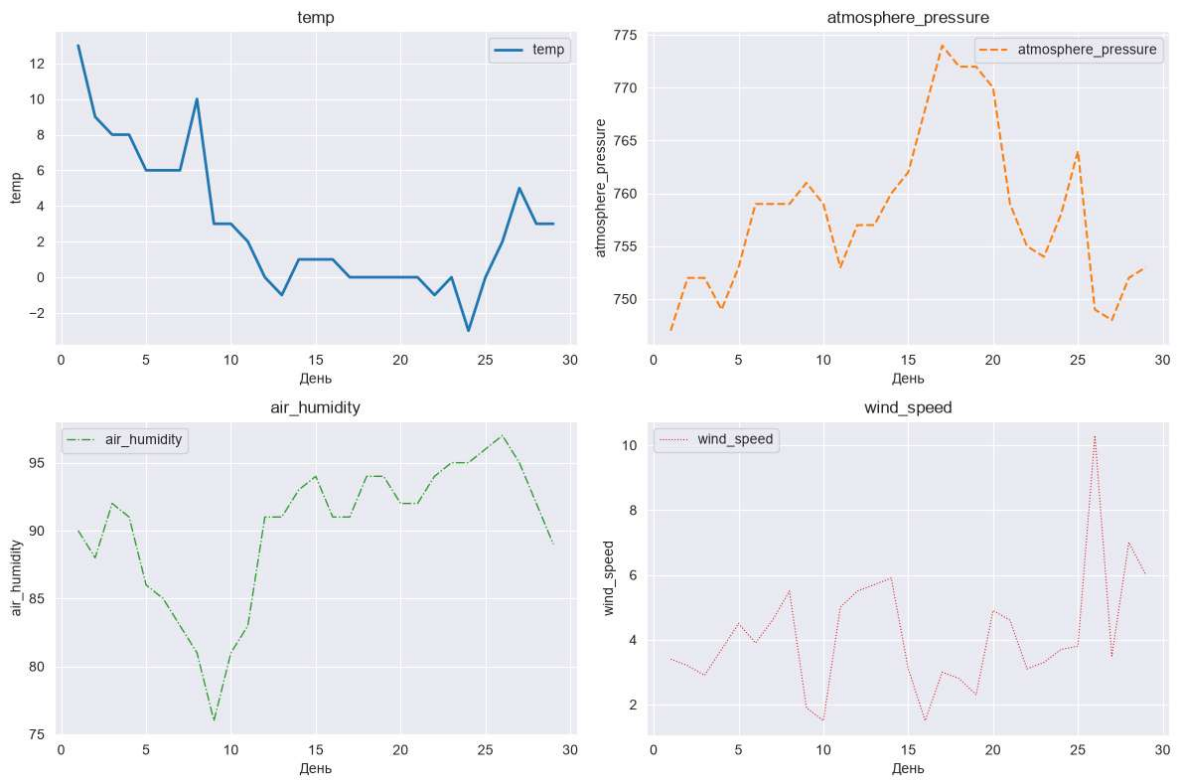
```
In [5]: # Решение задания 1
weather_data = pd.read_csv('pogoda.csv', sep='\t')

parameters = [column for column in weather_data.columns if column != 'day']
line_styles = ['-', '--', '-.', ':']
line_widths = [2, 1.5, 1, 0.8]
colors = plt.cm.tab10.colors

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

for index, parameter in enumerate(parameters):
    row, col = divmod(index, 2)
    axes[row, col].plot(weather_data['day'], weather_data[parameter],
                        color=colors[index % len(colors)],
                        linestyle=line_styles[index % len(line_styles)],
                        linewidth=line_widths[index % len(line_widths)],
                        label=parameter)
    axes[row, col].set_title(parameter)
    axes[row, col].set_xlabel('День')
    axes[row, col].set_ylabel(parameter)
    axes[row, col].legend()

plt.tight_layout()
plt.show()
```



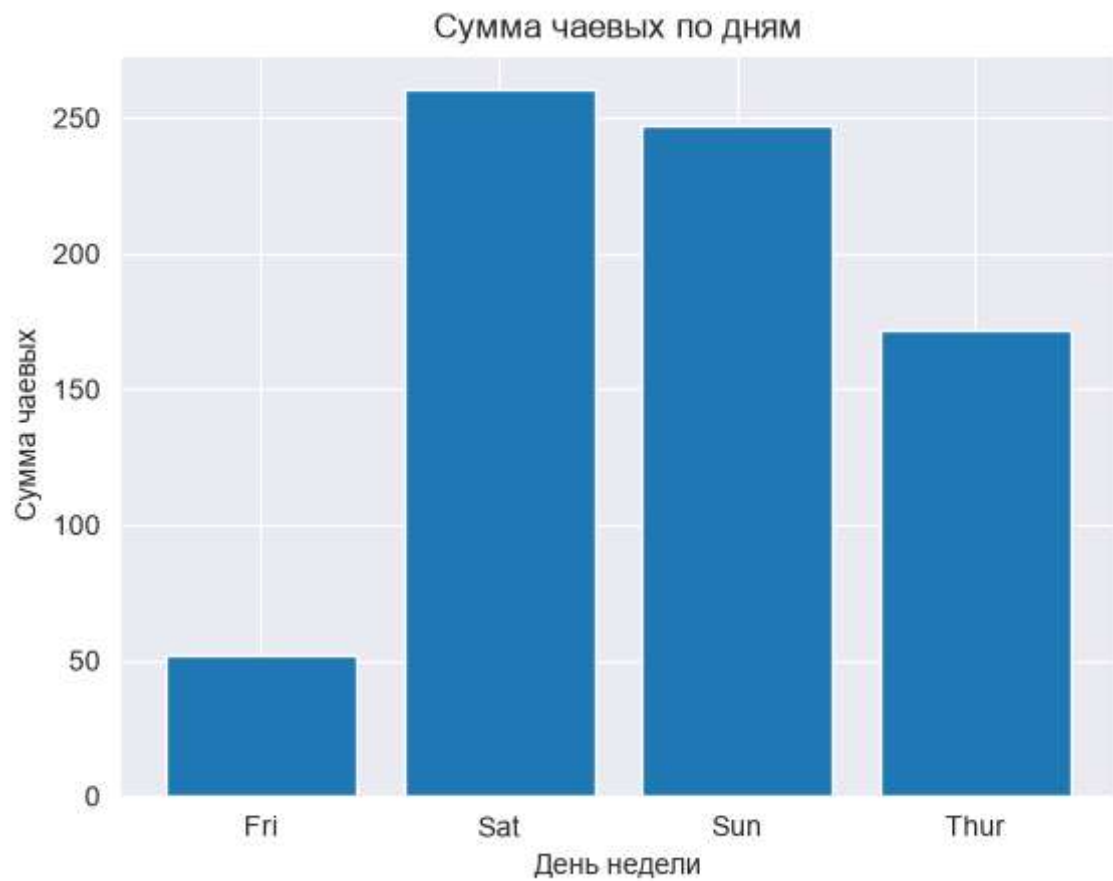
Задание 2. - Загрузите набор данных `tips.csv` . Выполните следующие задания:

1. С помощью библиотеки `matplotlib` постройте столбчатую диаграмму, где на оси X будут названия дней, а на оси Y сумма чаевых. Вам потребуется сгруппировать данные.
2. Сделайте так, чтобы названия дней располагались по диагонали.
3. Сделайте так, чтобы у каждого дня был свой уникальный цвет.

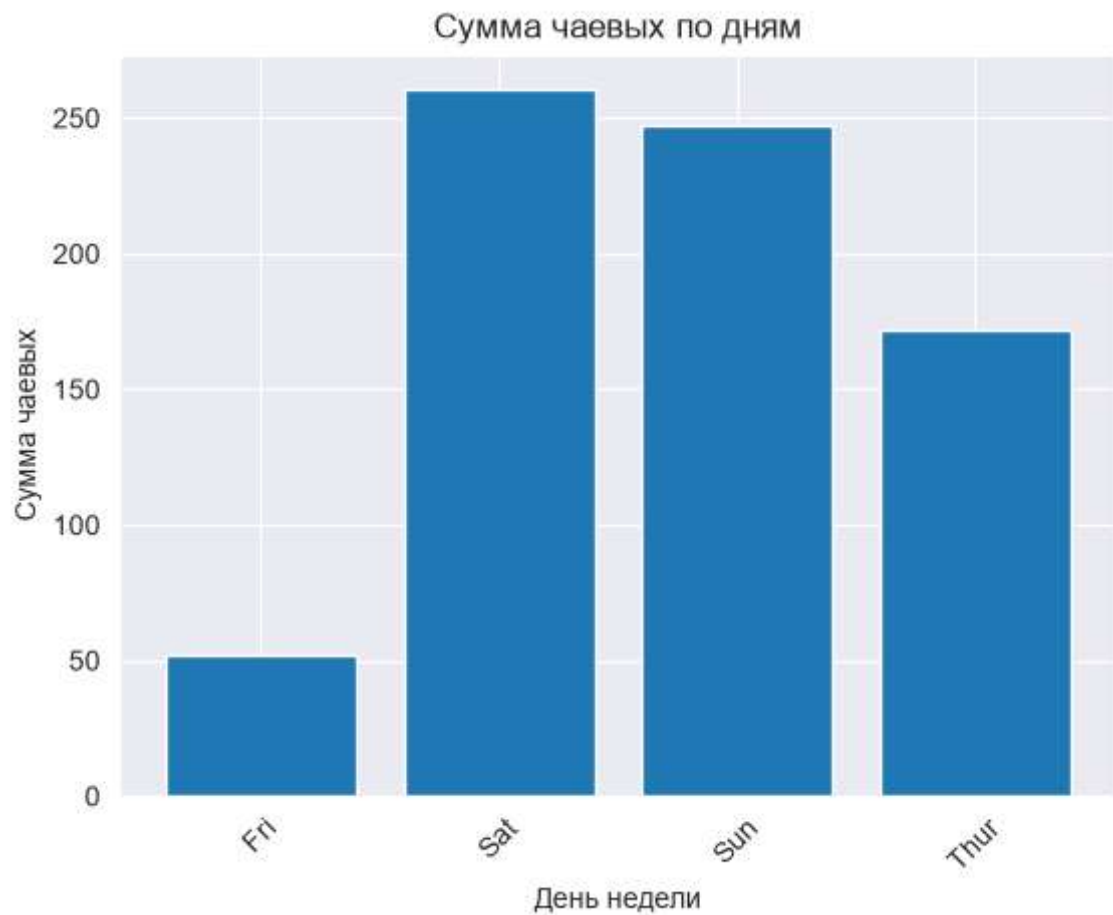
```
In [6]: # решение задания 2
# пункт 2.1
tips_data = pd.read_csv('tips.csv')

tips_by_day = tips_data.groupby('day')['tip'].sum()

plt.bar(tips_by_day.index, tips_by_day.values)
plt.xlabel('День недели')
plt.ylabel('Сумма чаевых')
plt.title('Сумма чаевых по дням')
plt.show()
```

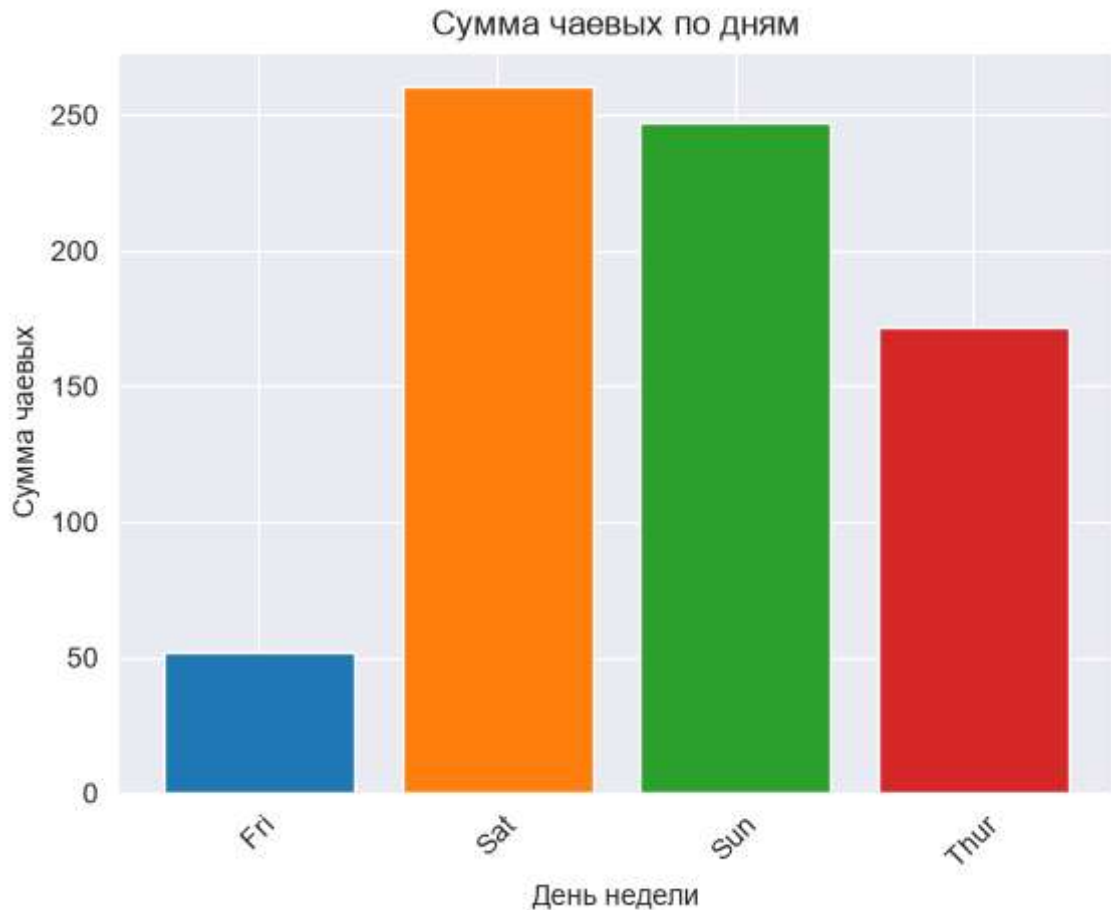


```
In [7]: # пункт 2.2
plt.bar(tips_by_day.index, tips_by_day.values)
plt.xlabel('День недели')
plt.ylabel('Сумма чаевых')
plt.title('Сумма чаевых по дням')
plt.xticks(rotation=45)
plt.show()
```



```
In [8]: # пункт 2.3
day_colors = plt.cm.tab10.colors[:len(tips_by_day)]

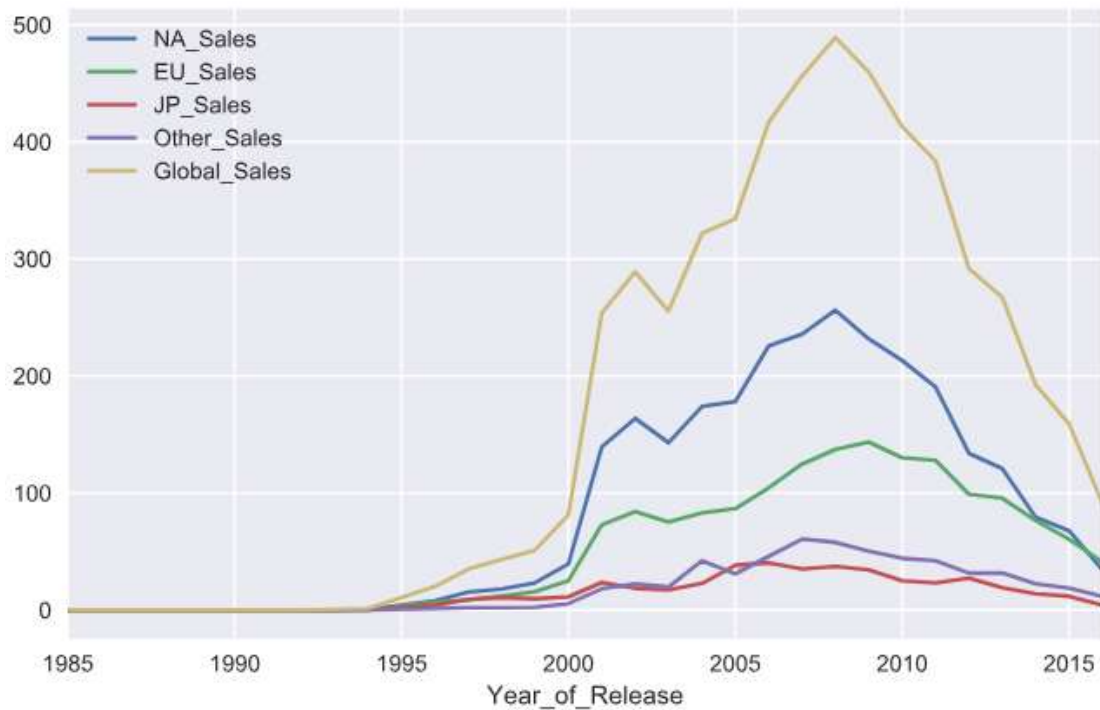
plt.bar(tips_by_day.index, tips_by_day.values, color=day_colors)
plt.xlabel('День недели')
plt.ylabel('Сумма чаевых')
plt.title('Сумма чаевых по дням')
plt.xticks(rotation=45)
plt.show()
```



Задание 3. Загрузите набор данных `Video_Games_Sales_as_at_22_Dec_2016.csv`

1. Удалите из него все записи с пропусками и выведите размеры датафрейма (должно получиться 6825 записи)
2. Удалите лишние параметры. У вас должны остаться: 'Name', 'Platform', 'Year_of_Release', 'Genre', 'NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales', 'Global_Sales', 'Critic_Score', 'Critic_Count', 'User_Score', 'User_Count', 'Rating'
3. Скорректируйте формат данных. Параметр 'User_Score' должен иметь формат 'float64', 'Year_of_Release' — 'int64', 'User_Count' — 'int64', 'Critic_Count' — 'int64'
4. Выведите полученную таблицу
5. По полученной таблице постройте график продаж видео игр в различных странах в зависимости от года (см. рис. 2).

Рисунок 2 - Примерный результат



```
In [9]: # решение задания 3
# пункт 1
games_data = pd.read_csv('Video_Games_Sales_as_at_22_Dec_2016.csv')
games_data = games_data.dropna()
print(games_data.shape)
```

(6825, 16)

```
In [10]: # пункт 3.2
columns_to_keep = ['Name', 'Platform', 'Year_of_Release', 'Genre', 'NA_Sales',
                  'EU_Sales', 'JP_Sales', 'Other_Sales', 'Global_Sales',
                  'Critic_Score', 'Critic_Count', 'User_Score', 'User_Count', '
games_data = games_data[columns_to_keep]
```

```
In [11]: # пункт 3.3
games_data['User_Score'] = games_data['User_Score'].astype('float64')
games_data['Year_of_Release'] = games_data['Year_of_Release'].astype('int64')
games_data['User_Count'] = games_data['User_Count'].astype('int64')
games_data['Critic_Count'] = games_data['Critic_Count'].astype('int64')
```

```
In [12]: # пункт 3.4
games_data
```

Out[12]:

	Name	Platform	Year_of_Release	Genre	NA_Sales	EU_Sales	JP_Sales	Other_Sales
0	Wii Sports	Wii	2006	Sports	41.36	28.96	3.77	
2	Mario Kart Wii	Wii	2008	Racing	15.68	12.76	3.79	
3	Wii Sports Resort	Wii	2009	Sports	15.61	10.93	3.28	
6	New Super Mario Bros.	DS	2006	Platform	11.28	9.14	6.50	
7	Wii Play	Wii	2006	Misc	13.96	9.18	2.93	
...	...	...	...	...	...	...	...	...
16667	E.T. The Extra-Terrestrial	GBA	2001	Action	0.01	0.00	0.00	
16677	Mortal Kombat: Deadly Alliance	GBA	2002	Fighting	0.01	0.00	0.00	
16696	Metal Gear Solid V: Ground Zeroes	PC	2014	Action	0.00	0.01	0.00	
16700	Breach	PC	2011	Shooter	0.01	0.00	0.00	
16706	STORM: Frontline Nation	PC	2011	Strategy	0.00	0.01	0.00	

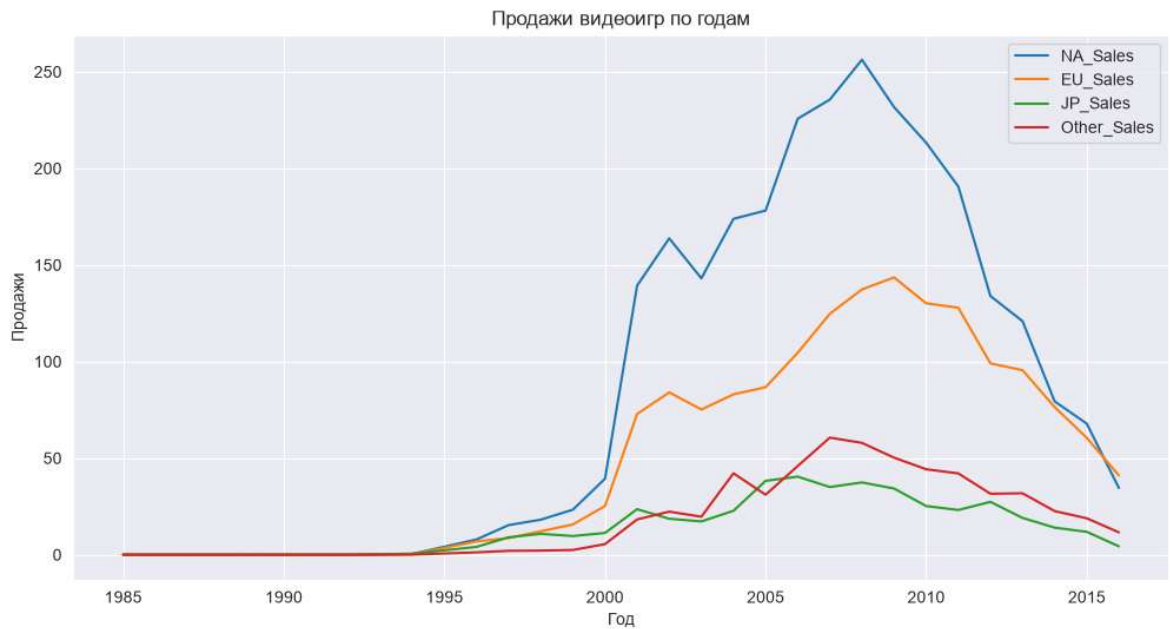
6825 rows × 14 columns



In [13]:

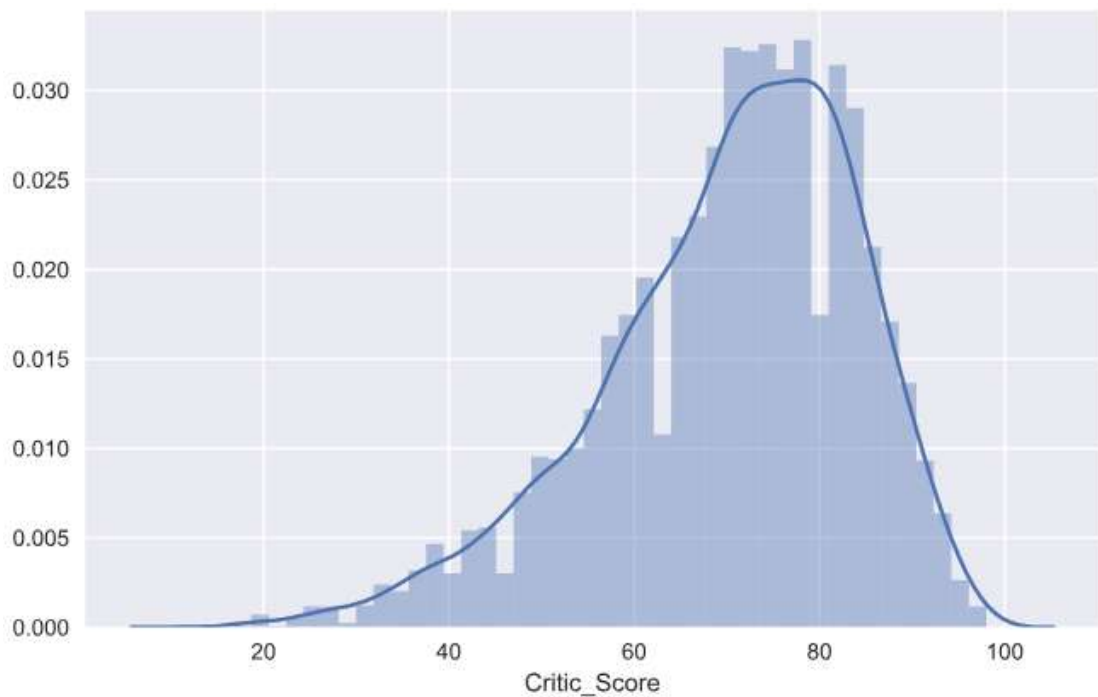
```
# пункт 3.5
sales_by_year = games_data.groupby('Year_of_Release')[['NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales']]

plt.figure(figsize=(12, 6))
plt.plot(sales_by_year.index, sales_by_year['NA_Sales'], label='NA_Sales')
plt.plot(sales_by_year.index, sales_by_year['EU_Sales'], label='EU_Sales')
plt.plot(sales_by_year.index, sales_by_year['JP_Sales'], label='JP_Sales')
plt.plot(sales_by_year.index, sales_by_year['Other_Sales'], label='Other_Sales')
plt.xlabel('Год')
plt.ylabel('Продажи')
plt.title('Продажи видеоигр по годам')
plt.legend()
plt.show()
```

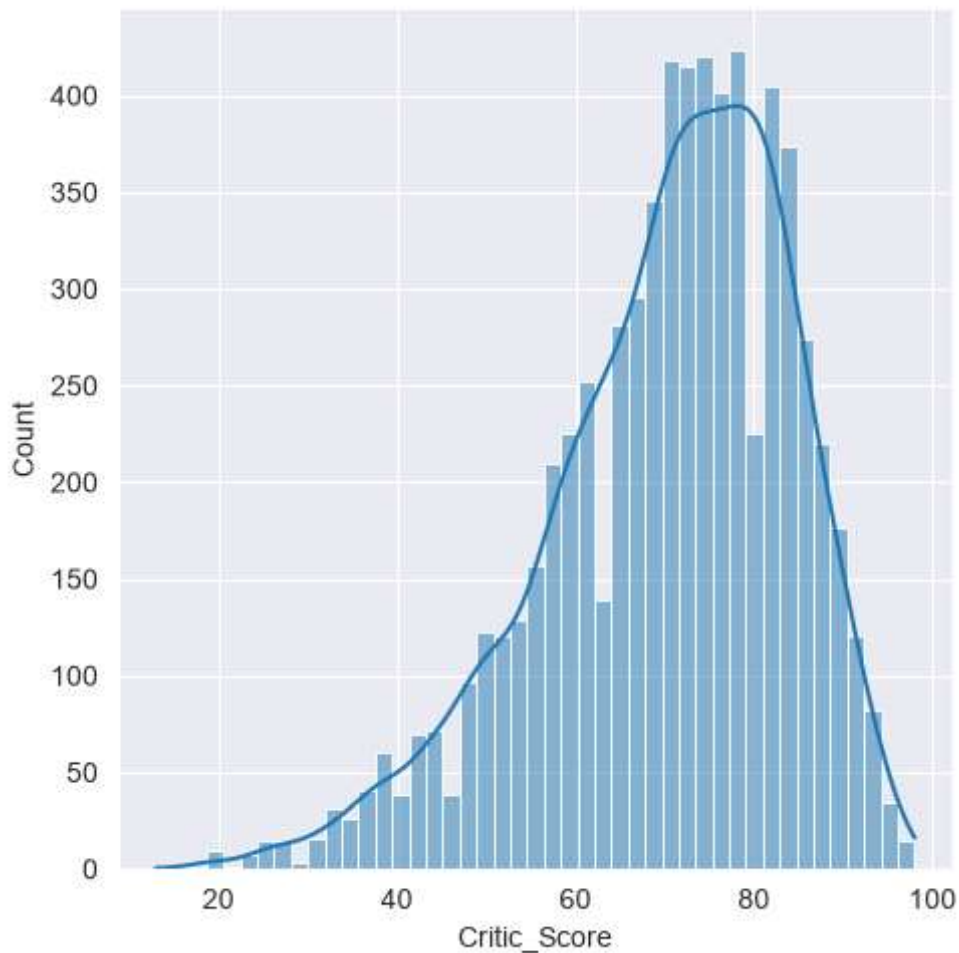


Задание 4. С помощью библиотеки `seaborn` постройте распределение оценок критиков `Critic_Score` (рисунок 3) для датасета, загруженного в задании 3.

Рисунок 3 - Примерный результат

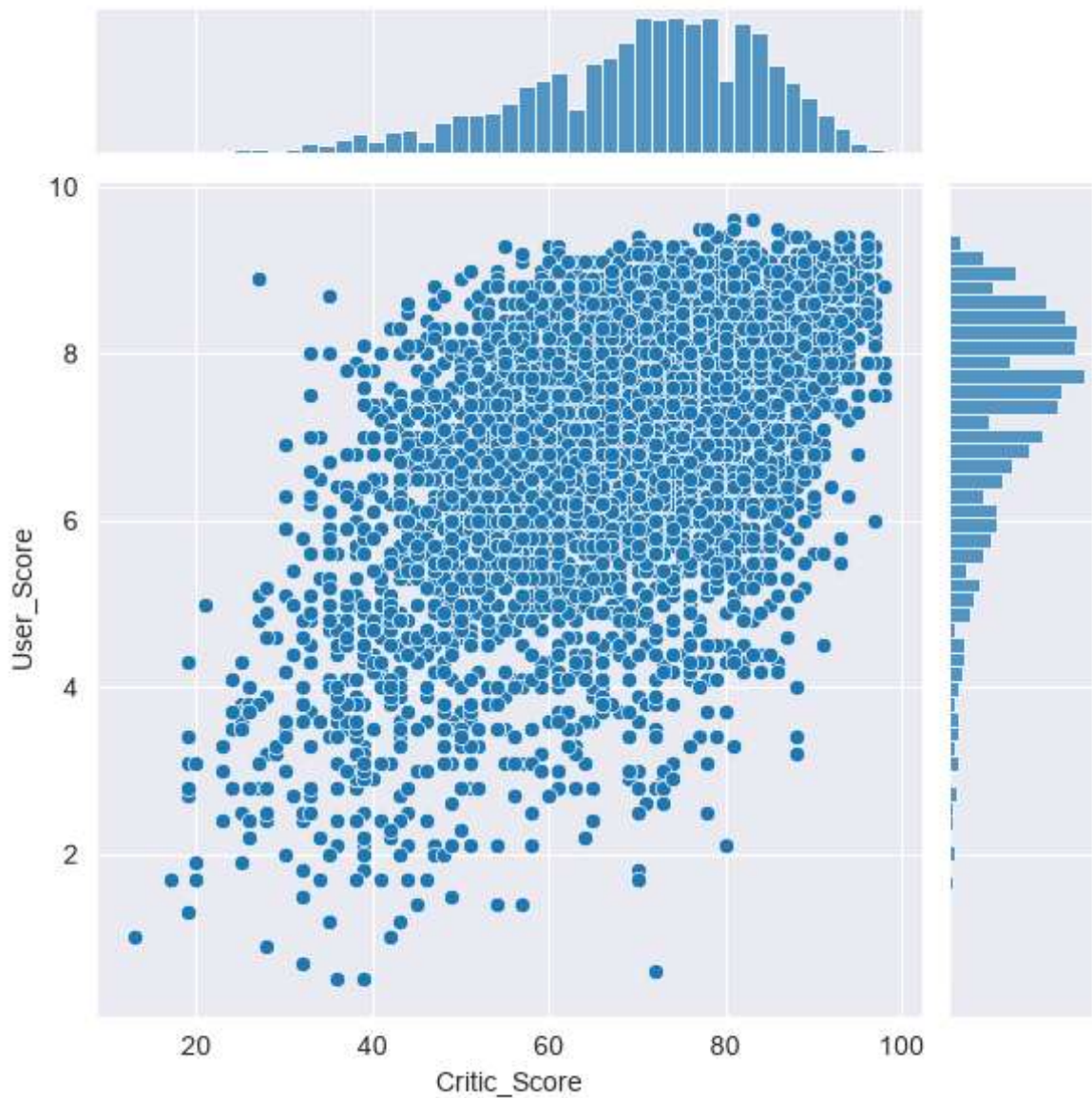


```
In [14]: # решение задания 4
sns.displot(games_data['Critic_Score'], kde=True)
plt.show()
```



Задание 5. Для того, чтобы подробнее посмотреть на взаимосвязь двух численных признаков, можно использовать `joint_plot` — это гибрид `scatter plot` и `histogram` (отображаются также гистограммы распределений признаков). Продемонстрируйте, как связаны между собой оценка критиков `Critic_Score` и оценка пользователя `User_Score` для датасета из задания 3.

```
In [15]: # решение задания 5
sns.jointplot(data=games_data, x='Critic_Score', y='User_Score')
plt.show()
```



Задание 6. Для датасета из задания 3 постройте pair plot для следующих признаков: 'Global_Sales', 'Critic_Score', 'Critic_Count', 'User_Score', 'User_Count'

```
In [16]: # решение задания 6
pairplot_columns = ['Global_Sales', 'Critic_Score', 'Critic_Count', 'User_Score']
sns.pairplot(games_data[pairplot_columns])
plt.show()
```

